ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 27

# APIs REST con PHP puro

- Qué es una API REST
- Verbos y códigos de estado
- Responder en JSON
- Recibir JSON: php://input
- Las rutas de la API
- El controlador de la API
- Proteger la API con un token
- Errores en JSON
- Probar la API
- Consumir APIs con cURL



**Ramón Abramo**

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

# 🧭 Contenido de la clase

Hasta ahora nuestra web hablaba con **personas**: devolvía HTML para que lo viera alguien en un navegador. Pero hoy casi ninguna aplicación vive sola. Una app móvil necesita la lista de cursos; una web hecha en React quiere los mismos datos; la academia quiere cobrar con una pasarela de pago o consultar el tiempo para una excursión. Ahí ya no hablan personas: hablan **programas**. Y el idioma que usan es **JSON** a través de una **API**.

Hoy terminamos nuestro MVC dándole una **API REST** de cursos, protegida con un token, y aprendemos a **consumir** APIs desde PHP con **cURL**. Con esta clase cerramos la parte del *framework* propio. Lo siguiente ya es Laravel.


•
👌 Cómo vamos a trabajar



1
📘 Qué es una API REST



2
🤔 Verbos y códigos de estado



3
👍 Responder en JSON



4
🔍 Recibir JSON: php://input



5
👌 Las rutas de la API



6
🚀 El controlador de la API



7
⚠️ Proteger la API con un token



8
🐞 Errores en JSON



9
😮 Probar la API



10
🔍 Consumir APIs con cURL



11
🚀 Ejercicios



12
👋 Lo que tenéis que llevaros de hoy





CLASE 27

## 👌 Cómo vamos a trabajar

Sustituid el contenido de `academia-mvc` por el de `proyecto/` y ejecutad `composer dump-autoload`. Añadid a vuestro `.env` las dos variables nuevas de `.env.ejemplo`:

Esquema

```
API_TOKEN=cambiad-este-token-por-uno-largo-y-secreto
API_URL=http://academia-mvc.test
```

Esquema

```
src/Controladores/Api/CursosApiControlador.php   ← NUEVO
src/Middleware/TokenApi.php                      ← NUEVO
src/Servicios/ClienteApi.php                     ← NUEVO: cliente cURL
herramientas/cliente-api.php                     ← NUEVO: lo probamos desde la consola
src/Nucleo/Peticion, Respuesta, Router y ManejadorErrores, ampliados
src/Modelos/Curso.php                            ← ahora sabe convertirse en JSON
public/.htaccess                                 ← una regla nueva para la cabecera
Authorization
```



CLASE 27 · PUNTO 1

## 📘 Qué es una API REST

Una **API** (*Application Programming Interface*) es una puerta pensada para **programas**, no para personas. En lugar de una página con estilos y botones, devuelve **datos** en un formato que otro programa entiende: hoy, casi siempre, **JSON**.

Esquema

```
Web (para personas)                  API (para programas)
GET /cursos/2                        GET /api/cursos/2
─────────────                        ──────────────────
<h1>🎓 JavaScript</h1>               { "datos": { "id": 2,
<ul><li>Duración: 60 horas</li>…                  "titulo": "JavaScript",
                                                  "horas": 60, … } }
```

**REST** es la forma más extendida de diseñar estas API [1]. Se resume en tres ideas:

| Idea | En nuestra API |
|---|---|
| Todo es un **recurso** con su dirección | `/api/cursos` (la colección) y `/api/cursos/2` (uno concreto) |
| El **verbo** HTTP dice qué hacer con él | `GET` leer, `POST` crear, `PUT` actualizar, `DELETE` borrar |
| El **código de estado** dice cómo ha ido | `200`, `201`, `404`, `422`… |

Fijaos en que es el mismo CRUD de siempre, pero con los verbos HTTP «de verdad» en lugar de rutas como `/borrar`.



CLASE 27 · PUNTO 2

## 🤔 Verbos y códigos de estado

| Verbo | Ruta | Acción | Respuesta si todo va bien |
|---|---|---|---|
| `GET` | `/api/cursos` | Listar (con `?q=` y `?pagina=`) | **200** OK + la lista |
| `GET` | `/api/cursos/{id}` | Ver uno | **200** OK + el curso |
| `POST` | `/api/cursos` | Crear | **201** Created + el curso + cabecera `Location` |
| `PUT` | `/api/cursos/{id}` | Actualizar | **200** OK + el curso actualizado |
| `DELETE` | `/api/cursos/{id}` | Borrar | **204** No Content (sin cuerpo) |

Y los códigos de error que usa nuestra API [2]:

| Código | Significa | Cuándo |
|---|---|---|
| **401** Unauthorized | Falta identificarse | Escribir sin token o con un token incorrecto |
| **404** Not Found | No existe | `/api/cursos/99` |
| **405** Method Not Allowed | La ruta no admite ese verbo | `PATCH /api/cursos/2` |
| **409** Conflict | Choca con el estado de los datos | Borrar un curso que tiene alumnos |
| **422** Unprocessable Content | Datos no válidos | Un título de dos letras |
| **500** Internal Server Error | Fallo nuestro | Una excepción no controlada |



🎯 En una API, el código de estado **es** la respuesta. El programa que os llama lo mira antes que nada: si es 201, sabe que se ha creado; si es 422, sabe que tiene que mostrar los errores.



CLASE 27 · PUNTO 3

# 👍 Responder en JSON

Añadimos a `Respuesta` un método que convierte cualquier dato en JSON y pone la cabecera correcta:

Respuesta.php — php-paso-a-paso/clase27 — Visual Studio Code

Respuesta.php

```php
<?php
public static function json(mixed $datos, int $codigo = 200, array $cabeceras = []):
Respuesta
{
    $json = json_encode($datos, JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES |
JSON_PRETTY_PRINT | JSON_THROW_ON_ERROR);
    return new Respuesta($json, $codigo, ["Content-Type" => "application/json; charset=utf-
8"] + $cabeceras);
}
```

¿Y cómo sabe `json_encode()` convertir un objeto `Curso`? Se lo decimos nosotros implementando la interfaz `JsonSerializable` de PHP [3]:

Respuesta.php — php-paso-a-paso/clase27 — Visual Studio Code

Respuesta.php

```php
<?php
class Curso extends Modelo implements JsonSerializable
{
    public function jsonSerialize(): array
    {
        return [
            "id" => $this->id,
            "titulo" => $this->titulo,
            "horas" => $this->horas,
            "precio" => $this->precio,
            "precio_con_iva" => $this->precioConIva(),
            "plazas" => $this->plazas,
            "completo" => $this->estaCompleto(),
        ];
    }
}
```

Así decidimos qué se publica: podemos añadir datos calculados ( `precio_con_iva` ) y, sobre todo, **no** publicar lo que no debe salir. Pensad en un `Usuario` : su `clave_hash` jamás debería aparecer en una API.

📄 Archivos: `src/Nucleo/Respuesta.php` y `src/Modelos/Curso.php`



💡 Los nombres de las claves del JSON van en minúsculas con guion bajo ( `precio_con_iva` ), como las columnas de la base de datos. Es lo más habitual en las API y lo que hace Laravel.



CLASE 27 · PUNTO 4

## 🔍 Recibir JSON: php://input

Cuando un programa **envía** datos a la API, no usa un formulario: manda el JSON en el **cuerpo** de la petición, con la cabecera `Content-Type: application/json`. PHP no lo pone en `$_POST`; hay que leerlo del flujo especial `php://input` [4]:

Peticion.php — php-paso-a-paso/clase27 — Visual Studio Code

Peticion.php

```php
<?php
$datos = $_POST;
if (str_contains($cabeceras["content-type"], "application/json")) {
    $datos = json_decode(file_get_contents("php://input") ?: "[]", true) ?? [];
}
```

Lo hacemos dentro de `Peticion::desdeGlobales()`, y el resultado va al mismo sitio que los datos de un formulario. Gracias a eso, `$peticion->validar()` **funciona igual** para la web y para la API: las mismas reglas, el mismo código.

📄 Archivo: `src/Nucleo/Peticion.php`



CLASE 27 · PUNTO 5

## 👌 Las rutas de la API

El router aprende dos verbos nuevos, `put()` y `delete()`, y las rutas de la API van todas bajo `/api`:

rutas.php — php-paso-a-paso/clase27 — Visual Studio Code

rutas.php

```php
<?php
// Leer es público
$router->get("/api/cursos", [CursosApiControlador::class, "index"]);
$router->get("/api/cursos/{id:\d+}", [CursosApiControlador::class, "ver"]);

// Escribir exige el token
$router->grupo([TokenApi::class], function (Router $router): void {
    $router->post("/api/cursos", [CursosApiControlador::class, "guardar"]);
    $router->put("/api/cursos/{id:\d+}", [CursosApiControlador::class, "actualizar"]);
    $router->delete("/api/cursos/{id:\d+}", [CursosApiControlador::class, "borrar"]);
});
```

📄 Archivo: `config/rutas.php`



⚠️ **¿Y el CSRF?** El middleware `VerificarCsrf` deja pasar las rutas `/api/…`. El ataque CSRF aprovecha que el navegador **envía solo la cookie de sesión**; nuestra API no usa cookies, sino un token que el programa tiene que añadir a mano en cada petición. Sin cookie, no hay CSRF que aprovechar.



CLASE 27 · PUNTO 6

## 🚀 El controlador de la API

Es el mismo CRUD de la clase 26, pero devolviendo JSON. Mirad `guardar()` y `borrar()`:

CursosApiControlador.php — php-paso-a-paso/clase27 — Visual Studio Code

CursosApiControlador.php

```php
<?php
// POST /api/cursos
public function guardar(Peticion $peticion): Respuesta
{
    $datos = $peticion->validar(Curso::reglas());

    $curso = (new Curso())->rellenar($datos);
    $curso->guardar();

    return Respuesta::json(["datos" => $curso], 201, ["Location" => "/api/cursos/{$curso
->id}"]);
}

// DELETE /api/cursos/{id}
public function borrar(Peticion $peticion, int $id): Respuesta
{
    $curso = Curso::buscar($id) ?? $this->noEncontrado("No existe ningún curso con el id
{$id}.");

    try {
        $curso->borrar();
    } catch (\PDOException $e) {
        if (($e->errorInfo[1] ?? 0) !== 1451) {
            throw $e;
        }
        throw new HttpException(409, "No se puede borrar: el curso tiene alumnos.");
    }

    return Respuesta::vacia();   // 204: borrado, y nada que contar
}
```

Y el listado devuelve, además de los datos, información para paginar:

datos.json — php-paso-a-paso/clase27 — Visual Studio Code

datos.json

```json
{
    "datos": [ { "id": 9, "titulo": "Linux para desarrolladores", … } ],
    "meta": { "pagina": 2, "por_pagina": 5, "total": 8, "paginas": 2 }
}
```

📄 Archivo: `src/Controladores/Api/CursosApiControlador.php`



💡 ¿Por qué envolver los datos en `"datos"` en lugar de devolver la lista directamente? Para poder añadir más cosas al lado (como `"meta"`) sin romper a quien ya usa la API. Laravel hace lo mismo con su clave `data`.



CLASE 27 · PUNTO 7

# ⚠️ Proteger la API con un token

Los programas no rellenan formularios de login. Se identifican enviando un **token** (una clave larga y secreta) en una cabecera de cada petición:

Esquema

```
Authorization: Bearer cambiad-este-token-por-uno-largo-y-secreto
```

El middleware `TokenApi` lo compara con el del `.env`:

TokenApi.php — php-paso-a-paso/clase27 — Visual Studio Code

TokenApi.php

```php
<?php
class TokenApi implements Middleware
{
    public function procesar(Peticion $peticion, callable $siguiente): Respuesta
    {
        $tokenEsperado = Entorno::obtener("API_TOKEN");

        if ($tokenEsperado === "" || !hash_equals($tokenEsperado, $peticion->tokenBearer()))
{
            throw new HttpException(401, "Falta el token de la API o no es válido.");
        }
        return $siguiente($peticion);
    }
}
```

📄 Archivos: `src/Middleware/TokenApi.php` y `public/.htaccess`



⚠️ **Detalle de Apache:** por seguridad, Apache no le pasa a PHP la cabecera `Authorization` a menos que se lo pidáis. Por eso el `.htaccess` lleva una regla nueva. Si vuestra API siempre responde 401 aunque el token esté bien, revisad esto primero.



🎯 Un único token para toda la API está bien para aprender. En una aplicación real, cada usuario o cada aplicación tiene **su propio** token, guardado en la base de datos y que se puede revocar. Es lo que hace Laravel Sanctum.



CLASE 27 · PUNTO 8

# 🐞 Errores en JSON

Si un programa pide JSON y recibe la página HTML del error 404, no sabrá qué hacer con ella. El manejador de errores mira si la petición era para `/api/` y, en ese caso, responde en JSON:

ManejadorErrores.php — php-paso-a-paso/clase27 — Visual Studio Code

ManejadorErrores.php

```php
<?php
private static function manejarApi(Throwable $e): void
{
    [$codigo, $cuerpo] = match (true) {
        $e instanceof ValidacionException ⇒ [422, ["error" ⇒ "Los datos no son válidos.",
"errores" ⇒ $e→errores]],
        $e instanceof HttpException ⇒ [$e→codigo, ["error" ⇒ $e→getMessage()]],
        default ⇒ [500, ["error" ⇒ "Error del servidor."]],
    };
    Respuesta::json($cuerpo, $codigo)→enviar();
}
```

Así, un error de validación en la API no redirige a ninguna parte: devuelve un **422** con los errores de cada campo, listos para que la app los muestre.

datos.json — php-paso-a-paso/clase27 — Visual Studio Code

datos.json

```json
{
    "error": "Los datos no son válidos.",
    "errores": {
        "titulo": "Debe tener al menos 3 caracteres.",
        "horas": "Debe ser como mínimo 1."
    }
}
```

📄 Archivo: `src/Nucleo/ManejadorErrores.php`



CLASE 27 · PUNTO 9

# 😮 Probar la API

Las peticiones `GET` se pueden probar en el navegador: abrid `http://academia-mvc.test/api/cursos`. Para el resto necesitáis una herramienta que envíe POST, PUT o DELETE con cabeceras y cuerpo JSON:

| Herramienta | Dónde |
|---|---|
| **Thunder Client** | Extensión de VS Code: una pestaña más del editor |
| **Postman** | Aplicación de escritorio, la más conocida |
| **curl** | Línea de comandos; viene con Windows 10 y 11 |

Con `curl`, desde la Terminal de Laragon:

terminal — php-paso-a-paso/clase27 — Visual Studio Code

terminal

```
1 curl http://academia-mvc.test/api/cursos/2
```

terminal — php-paso-a-paso/clase27 — Visual Studio Code

terminal

```
1 curl -X POST http://academia-mvc.test/api/cursos -H "Content-Type: application/json" -H
  "Authorization: Bearer cambiad-este-token-por-uno-largo-y-secreto" -d "
  {\"titulo\":\"Vue.js\",\"horas\":40,\"precio\":150.5,\"plazas\":10}"
```

terminal — php-paso-a-paso/clase27 — Visual Studio Code

terminal

```
1 curl -X DELETE http://academia-mvc.test/api/cursos/13 -H "Authorization: Bearer cambiad-
  este-token-por-uno-largo-y-secreto"
```

## ✅ Pruebas para comprobarlo

| Petición | Debéis obtener |
|---|---|
| `GET /api/cursos?q=a&pagina=2` | 200, los cursos y el bloque `meta` |
| `GET /api/cursos/99` | 404 con `{"error": …}` |
| `POST /api/cursos` sin token | 401 |
| `POST /api/cursos` con token y un título de dos letras | 422 con los errores de cada campo |
| `POST /api/cursos` con datos correctos | 201, el curso nuevo y la cabecera `Location` |
| `PUT /api/cursos/{id}` | 200 con los datos cambiados |
| `DELETE /api/cursos/1` (tiene alumnos) | 409 |
| `DELETE` del curso que habéis creado | 204, sin cuerpo |
| `PATCH /api/cursos/2` | 405 |




Ahora damos la vuelta a la tortilla: nuestro PHP es quien **llama** a una API. Para eso PHP trae la extensión **cURL** [5]. La envolvemos en una clase para no repetir sus opciones en cada llamada:

ClienteApi.php — php-paso-a-paso/clase27 — Visual Studio Code

ClienteApi.php

```php
<?php
private function peticion(string $metodo, string $ruta, ?array $datos = null): array
{
    $cabeceras = ["Accept: application/json"];
    if ($this->token !== "") {
        $cabeceras[] = "Authorization: Bearer {$this->token}";
    }

    $curl = curl_init(rtrim($this->urlBase, "/") . $ruta);
    curl_setopt($curl, CURLOPT_RETURNTRANSFER, true);      // devuélveme la respuesta, no la imprimas
    curl_setopt($curl, CURLOPT_CUSTOMREQUEST, $metodo);    // GET, POST, PUT, DELETE…
    curl_setopt($curl, CURLOPT_TIMEOUT, $this->segundosMaximos);
    if ($datos !== null) {
        $cabeceras[] = "Content-Type: application/json";
        curl_setopt($curl, CURLOPT_POSTFIELDS, json_encode($datos));
    }
    curl_setopt($curl, CURLOPT_HTTPHEADER, $cabeceras);

    $cuerpo = curl_exec($curl);
    $codigo = (int) curl_getinfo($curl, CURLINFO_HTTP_CODE);
    curl_close($curl);

    return ["codigo" => $codigo, "datos" => json_decode($cuerpo, true) ?? []];
}
```

| Opción de cURL | Para qué |
|---|---|
| `CURLOPT_RETURNTRANSFER` | Que `curl_exec()` **devuelva** la respuesta en lugar de imprimirla |
| `CURLOPT_CUSTOMREQUEST` | El verbo HTTP |
| `CURLOPT_TIMEOUT` | Segundos máximos de espera. **Ponedlo siempre**: si la otra API se cuelga, que no se cuelgue vuestra web |
| `CURLOPT_HTTPHEADER` | Las cabeceras (token, tipo de contenido…) |
| `CURLOPT_POSTFIELDS` | El cuerpo de la petición |

Y con la clase, usar una API es muy cómodo:

ClienteApi.php — php-paso-a-paso/clase27 — Visual Studio Code

ClienteApi.php

```php
<?php
$api = new ClienteApi("http://academia-mvc.test", Entorno::obtener("API_TOKEN"));

$lista = $api->get("/api/cursos?q=php");
$creado = $api->post("/api/cursos", ["titulo" => "Curso de prueba", "horas" => 10, "precio" => 50, "plazas" => 5]);
```

Probadlo desde la Terminal de Laragon, en la carpeta del proyecto: crea, edita y borra un curso usando vuestra propia API.

terminal — php-paso-a-paso/clase27 — Visual Studio Code

terminal

```
php herramientas/cliente-api.php
```

📄 Archivos: `src/Servicios/ClienteApi.php` y `herramientas/cliente-api.php`



💡 La misma clase sirve para **cualquier** API: el tiempo, una pasarela de pago, un servicio de envíos… Solo cambian la dirección, el token y lo que devuelve. El ejercicio 03 consume una API pública de pruebas.



CLASE 27 · PUNTO 11

## 🚀 Ejercicios

Las soluciones se copian dentro del proyecto, respetando sus carpetas.

- ☐ **01 · Alumnos de un curso.** Una ruta `GET /api/cursos/{id}/alumnos` que devuelva el curso, sus alumnos y el total. El modelo `Alumno` también debe implementar `JsonSerializable`. → `Alumno.php`, `AlumnosApiControlador.php` y `config/rutas.php`
- ☐ **02 · DELETE desde un formulario.** Los formularios HTML solo saben hacer GET y POST. Haced que un campo oculto `_metodo` con el valor `DELETE` «disfrace» el POST, y cambiad la ruta web de borrado a `DELETE /cursos/{id}`. → `Peticion.php`, `config/rutas.php` y `vistas/cursos/ver.php`
- ☐ **03 · Una API externa.** Con `ClienteApi`, mostrad por consola el nombre, el email y la ciudad de los usuarios de la API pública de pruebas JSONPlaceholder (`https://jsonplaceholder.typicode.com/users`). Controlad el caso de que no haya conexión. → `herramientas/usuarios-externos.php`



💡 Lo del ejercicio 02 es exactamente lo que hace Laravel con `@method('DELETE')` dentro de un formulario.



🏆 **Reto final del MVC:** añadid a la API la ruta `GET /api/estadisticas`, que devuelva cuántos cursos hay, cuántos están completos y el precio medio. Después, construid una página web sencilla en `public/` que la consulte con `fetch()` de JavaScript y muestre las cifras. Acabáis de hacer que vuestro *backend* alimente a un *frontend*: lo que hace media internet.



CLASE 27 · PUNTO 12

## 👋 Lo que tenéis que llevaros de hoy

| Idea | En una frase |
|---|---|
| API | Una puerta para programas: devuelve datos (JSON), no páginas |
| REST | Recursos con su dirección + verbos HTTP + códigos de estado |
| Verbos | `GET` leer, `POST` crear, `PUT` actualizar, `DELETE` borrar |
| Códigos | 200, 201, 204 si va bien; 401, 404, 405, 409, 422 y 500 si no |
| `Respuesta::json()` | `json_encode()` + `Content-Type: application/json` |
| `JsonSerializable` | El objeto decide qué publica en el JSON |
| `php://input` | Donde llega el JSON que nos envían |
| Token `Bearer` | Cómo se identifica un programa; sin cookies, sin CSRF |
| cURL | Para que nuestro PHP llame a otras API; siempre con tiempo máximo |

## 🎉 Fin de nuestro propio MVC

En cinco clases habéis construido, pieza a pieza, un *framework* con front controller, router, peticiones y respuestas, modelos, vistas, controladores, variables de entorno, contenedor de dependencias, middleware, validación, autenticación y API REST. Repasad la tabla comparativa de la clase 26: **todas** esas piezas os esperan en Laravel. La diferencia es que ahora sabéis qué hay debajo de cada una. 💪

# 📚 Fuentes y documentación oficial

1. MDN Web Docs: REST
2. MDN Web Docs: códigos de estado de respuesta HTTP
3. Manual de PHP: la interfaz JsonSerializable
4. Manual de PHP: php:// (php://input)
5. Manual de PHP: cURL